Predictive Scheduling of Speculative Tool Calls Under Shared API Quotas

Jack McClure (jackmccl@umich.edu) | Saitej Velaga (saitejv@umich.edu)
Angela Li (anqili@umich.edu)

Problem and research question

Agents can reduce waiting by starting a tool call before the main model confirms that it needs the result [1]. However, an unused call can consume a shared API allowance. Even after it finishes or is cancelled, that allowance may remain spent until the service replenishes it. Another agent can then be delayed despite an otherwise idle service. We focus on read-only tools with this quota behavior, such as a rate-limited search endpoint.

We ask whether scheduling speculation using agents' current progress can reduce task completion time under a shared quota by anticipating delays to future required calls. Our hypothesis is that this information helps most when demand is bursty and quota replenishment is slow relative to tool execution. Under light load or inaccurate predictions, a simple quota reserve may work equally well.

Related work and proposed contribution

Speculative Actions predicts and executes possible future actions [1]. PASTE coordinates speculative tool execution and inference scheduling [2]. SPAgent controls speculation using expected latency savings and inference-engine load [3]. These systems already establish that speculation must account for resource contention. A Stanford course project schedules concurrent agent requests under external rate limits using current-stage progress [4]. Usage Guard forecasts quota exhaustion and protects reserves [5].

We propose an admission rule that estimates the additional waiting caused by each optional call, including required calls that have not yet been submitted. It accounts for quota that cancellation cannot restore and for later calls avoided when a speculative result is reused. Unlike quota-reserve and current-capacity baselines, or schedulers that prioritize required calls, our rule weighs a speculative call's savings against its predicted delay to other agents' future required calls. This cross-agent admission decision is our systems contribution; Jev, Kev, and the history table only forecast demand. Evaluation will separate the rule's benefit from predictor quality.

System design and predictor integration

A Python gateway will track quota, pending calls, and each agent's observed progress: completed-call count, current execution phase, and time since its last completion. For each speculative candidate, it will compare two short simulations with the same sampled future calls: launch now, or leave it unlaunched. Both account for queued and predicted required calls, quota refills, and result reuse. A fixed horizon and sample budget will bound computation. The gateway will admit speculation only when predicted savings exceed added waiting across agents and the estimated increase in deadline-miss risk stays below a threshold. This estimated risk limit is not a guarantee under arbitrary arrivals.

For example, agent A's speculative search could consume the last quota slot just before agent B needs a required search. The scheduler can reject A's call when B's predicted delay outweighs A's expected savings, even though speculation benefits A alone.

Jev [6] and one pretrained Kev model [8] will answer the same bounded questions about upcoming search demand and whether a candidate will be needed. We will combine predictions with time-to-next-call distributions learned from development traces. A history-based transition table will provide a cheaper alternative. Held-out traces will test calibration; all candidate-generation, predictor, and scheduler latency and costs will count toward results.

For Jev and Kev, we will compare task-and-candidate context, added recent tool-call history, and added bounded result excerpts and progress signals. Inputs will contain only information observed by decision time. These ablations will identify which context helps each predictor and whether that improvement lowers task latency after context and inference overhead.

One fixed candidate generator, adapted from [1], will propose at most one concrete search call per agent step using observed state. Results will remain hidden until the main agent requests the same tool and arguments. Matching in-flight calls will be joined; completed results will replace the later invocation without charging quota twice. The core implementation will use one search endpoint and a fixed-window quota; continuous refill is a stretch goal.

Implementation and evaluation

We will implement the gateway with Python asyncio and FastAPI, use discrete-event replay for large experiments, and analyze results in Jupyter. Separate modules will handle quota accounting, prediction, admission policies, and logging. A local search service over a fixed HotpotQA document subset [7] will provide repeatable results. We will begin with 10-20 pilot traces and target 100-200 task traces, split by task into development and evaluation sets. Quotas, deadlines, and arrival bursts will be controlled experimental settings.

Every policy will receive the same arrivals, required-call traces, and tool durations; speculative policies will share the candidate generator. Requests become ready after their recorded dependencies finish, allowing scheduling and reuse to shift later request times. Predictions will see only information available at the current replay time. A smaller live-agent experiment will check replay fidelity and answer correctness. Scheduling sweeps will run on a CPU; model access, available hardware, and trace-generation cost will be checked in the pilot.

Baselines will include no speculation, PASTE-inspired current-capacity admission, a tuned fixed reserve, a reserve based on recent usage forecasts, and a progress-aware required-call scheduler adapted from [4]. These will be lightweight policies in the same gateway, not full reimplementations of prior systems. We will hold required-call ordering fixed when comparing admission rules, then test the adapted progress-aware ordering with the same candidates. An offline reference using known future calls will test the value of accurate prediction.

Primary outcomes are mean and p95 task completion time and required-call waiting. We will also report deadline completion, unused calls, per-agent waiting, and overhead. We will test 1, 2, 4, and 8 concurrent agents sharing a fixed quota, with one agent as a control, and run focused sweeps of arrivals, bursts, refill period, and prediction error. With predictions fixed, we will compare full admission against the same rule ignoring delay to other agents, isolating the cross-agent decision. With the scheduler fixed, we will compare Jev, Kev, and history; context ablations will remove progress and history fields. Settings will be tuned on development tasks. Paired held-out runs with repeated seeds and confidence intervals will report gains and regressions. Checks will verify quota accounting, exact matching, and no future information in policy inputs.

Scope and deliverables

The four-week core will retain the multi-agent scheduler, one search workload, fixed-window quotas, all three predictors, replay, and a small live validation. Week one will build replay, check exact-call reuse and model access, and compare reserves with the future-informed reference. Week two will implement admission and the history baseline; week three will integrate Jev/Kev and the limited context ablations; week four will complete experiments, live validation, and analysis. Work will be divided across gateway/replay, predictor integration, and evaluation. Predictor training and additional workloads are outside scope.

For the November 2/4 checkpoint, we aim to show the replay prototype and initial reserve/reference results. If accurate prediction offers little improvement, we will characterize where simpler reserves suffice. If Jev is unavailable, we will proceed with history and Kev and report the limitation. The final eight-page report will include sensitivity tests and failure cases, with code and step-by-step run instructions delivered by 1:00 p.m. EST on December 17.

References

[1] N. Ye et al. Speculative Actions: A Lossless Framework for Faster Agentic Systems. arXiv:2510.04371, 2025, revised 2026. arxiv.org/abs/2510.04371. Code: github.com/naimengye/speculative-action

[2] Y. Sui et al. Parallelizing Tool Execution and LLM Generation for Low-Latency Agent Serving. arXiv:2603.18897, 2026. arxiv.org/abs/2603.18897

[3] Z. Huang et al. Reducing Latency of LLM Search Agent via Speculation-based Algorithm-System Co-Design. arXiv:2511.20048, 2025. arxiv.org/abs/2511.20048

[4] B. Yalcinkaya, F. Henriksson, and A. De. DAG-Aware Request Scheduling for LLM Agent Workloads Under External Rate Limits. Stanford CS244c, 2026. www.scs.stanford.edu/26wi-cs244c/proj/dag_sched_agent.pdf

[5] Agent Layer. Usage Guard. Software repository. github.com/agent-layer-mcp/usage-guard

[6] TypeSafe AI. Jev documentation and Python primitives. docs.typesafe.ai/primitives

[7] Z. Yang et al. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. EMNLP 2018. hotpotqa.github.io

[8] J. Palmer. Kev. Software and pretrained decision models. github.com/jaredpalmer/kev